PayGuard: Short Engineering Report

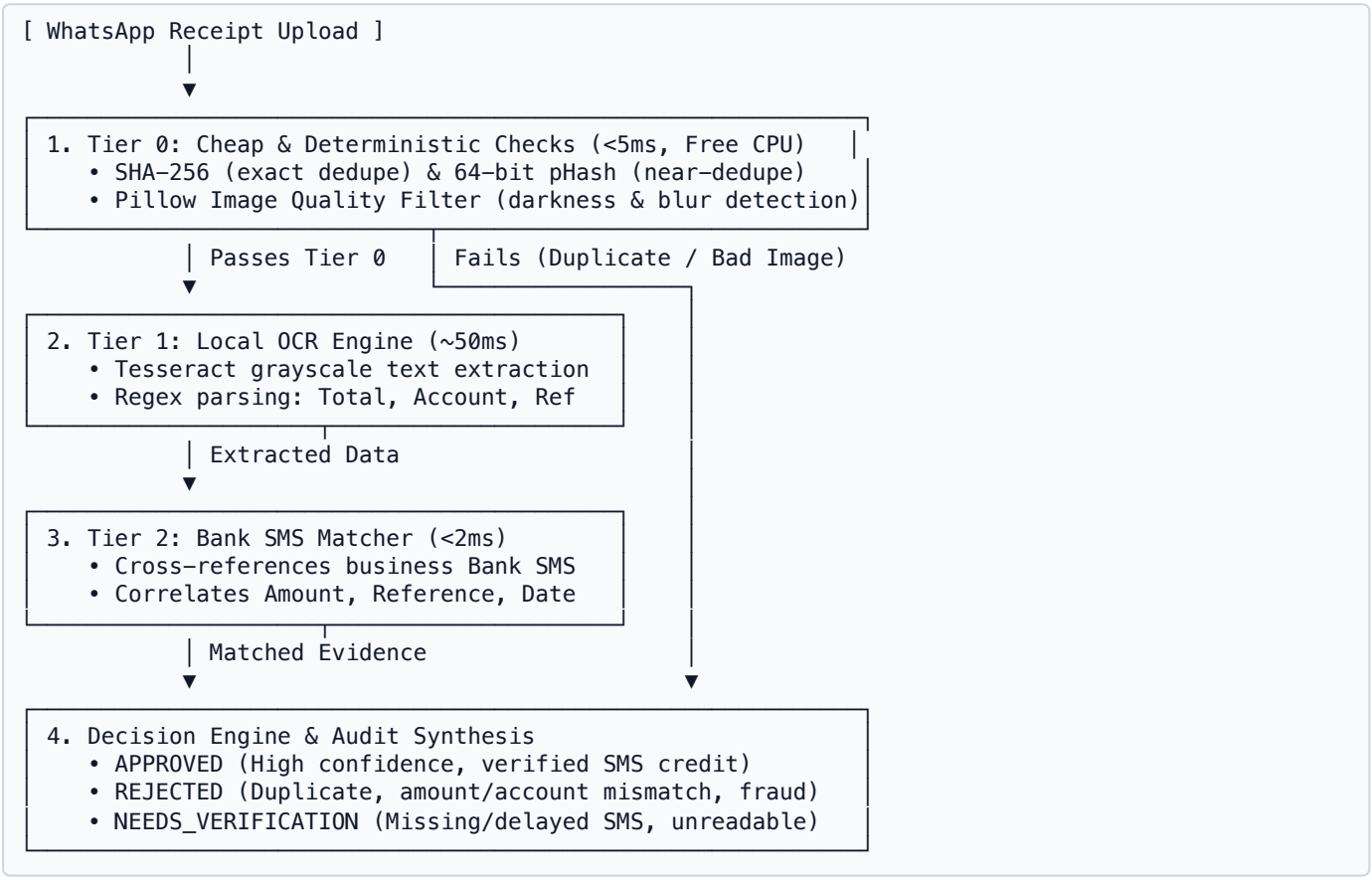
Automated Bank Payment Verification System | *BuildStart 2-Day Engineering Challenge*

Executive Summary

In Sri Lankan e-commerce, small merchants lack direct Open Banking APIs. When customers place orders via WhatsApp, they settle payments through manual bank transfers and submit screenshot receipts. Verifying these manually is slow and susceptible to image manipulation and reuse fraud. **PayGuard** is an automated, ultra-low-cost verification system that processes payment evidence through an escalating **3-Tier Pipeline** (Deterministic Hashing $\rightarrow$ Local OCR $\rightarrow$ Bank SMS Correlation). It delivers deterministic decisions (`APPROVED`, `REJECTED`, `NEEDS_VERIFICATION`) with near-zero compute cost and an explainable audit interface.

1. How the Solution Works

PayGuard implements an escalating state machine that filters payments by compute cost:



1. **Intake**: Customer submits a receipt image attached to an `order_id`.
2. **Tier 0**: Hashes and image quality metrics run first. Exact duplicates, cropped reuses, or dark slips terminate immediately without wasting OCR/AI compute.
3. **Tier 1**: Clean slips are parsed via local Tesseract OCR for numerical amounts, account numbers, and references.
4. **Tier 2**: Extracted parameters are correlated with the incoming bank credit SMS table.
5. **Decision Synthesis**: Outputs decision status, confidence score, internal merchant telemetry, and a safe customer WhatsApp reply.

2. Verifying a Payment Belongs to an Order

PayGuard binds transactions to specific orders using **multi-factor validation**: - **Receiving Account Match**: Extracted deposit account must match `Order.business_account_no`. Payments to personal or wrong accounts are rejected (`ACCOUNT_MISMATCH`). - **Strict Amount Equality**: Extracted total amount must strictly equal `Order.expected_amount_lkr`. Under/overpayments trigger `AMOUNT_MISMATCH`. - **Temporal Freshness Window**: Slip timestamp must fall after order creation and within a 30-day window (`max_payment_age_days = 30`). - **Reference Binding**: Bank SMS matching binds the bank credit reference to the specific `order_id`, preventing multiple orders from claiming the same deposit.

3. Detecting Duplicate and Reused Payments

PayGuard employs a dual-hash defense against duplication and screenshot tampering:

```
Incoming Image ──► SHA-256 Hash ──► Exact Match in DB? ──► [FLAG: EXACT_DUPLICATE -> REJECT]
                 └──► 64-bit pHash ──► Hamming Distance H(h1, h2) <= 6? ──► [FLAG: NEAR_DUPLICATE ->
```

- **Exact Duplicate Detection (`SHA-256`)**: Computes a 256-bit cryptographic digest of raw bytes in $<1\text{ms}$. Exact file matches are immediately rejected (`EXACT_DUPLICATE`).
- **Perceptual Near-Duplicate Detection (`pHash`)**: Computes a frequency-domain Discrete Cosine Transform (DCT) hash invariant to JPEG compression, minor cropping, or scaling. If the Hamming distance $\mathcal{H} \le 6$, it identifies the original submission ID and rejects reuse (`NEAR_DUPLICATE_IMAGE`).
- **Claimed Reference Tracking**: Transaction references already tied to a completed order cannot be reused for new orders.

4. Handling Suspicious or Manipulated Slips

- **Cross-Evidence Ground Truth**: An attacker editing an image (e.g. Photoshop-altered amount) **cannot forge the bank's incoming SMS notification**. Because approval requires Tier 2 SMS confirmation, manipulated slips fail to match real bank credits and route safely to `NEEDS_VERIFICATION`.
- **Reference Syntax Validation**: Genuine bank references follow strict alphanumeric patterns. Irregular or malformed references trigger `SUSPICIOUS_REFERENCE` and are rejected.
- **Pre-OCR Quality Assessment**: Degraded, pixelated, or excessively noisy images are flagged (`LOW_QUALITY_BLUR`, `LOW_QUALITY_DARK`) to prevent false OCR readings.

5. Bank SMS Ground Truth & Evidence Correlation

Bank SMS notifications serve as the **authoritative evidence** that funds arrived in the merchant's bank account.

```
Extracted Data ──► Exact Match (Amount + Ref No)? ──► Conf: 1.0 (High Trust Match)
                 └──► Fallback: Amount-Only Match?
                        ├── Single SMS found  ──► Conf: 0.6 (Amount Matched, Missing Ref)
                        └── Multiple SMS found ──► Conf: 0.2 (Ambiguous -> NEEDS_VERIFICATION)
```

- **Carrier Delays**: SMS notifications may lag customer slip uploads. When a slip is valid but the SMS has not yet arrived, PayGuard sets status to `NEEDS_VERIFICATION` (`NO_SMS_MATCH`) with a polite waiting message, rather than falsely rejecting genuine customers.
- **Multiple Similar Payments**: If several customers pay the same amount (e.g. Rs. 25,000) simultaneously, the system requires reference correlation or holds for human review to prevent assigning payments to the wrong order.

6. Cost Minimisation Architecture

Sending every image to commercial multimodal LLMs (e.g. GPT-4o @ ~$0.02/image) creates unsustainable unit economics for low-margin retail.

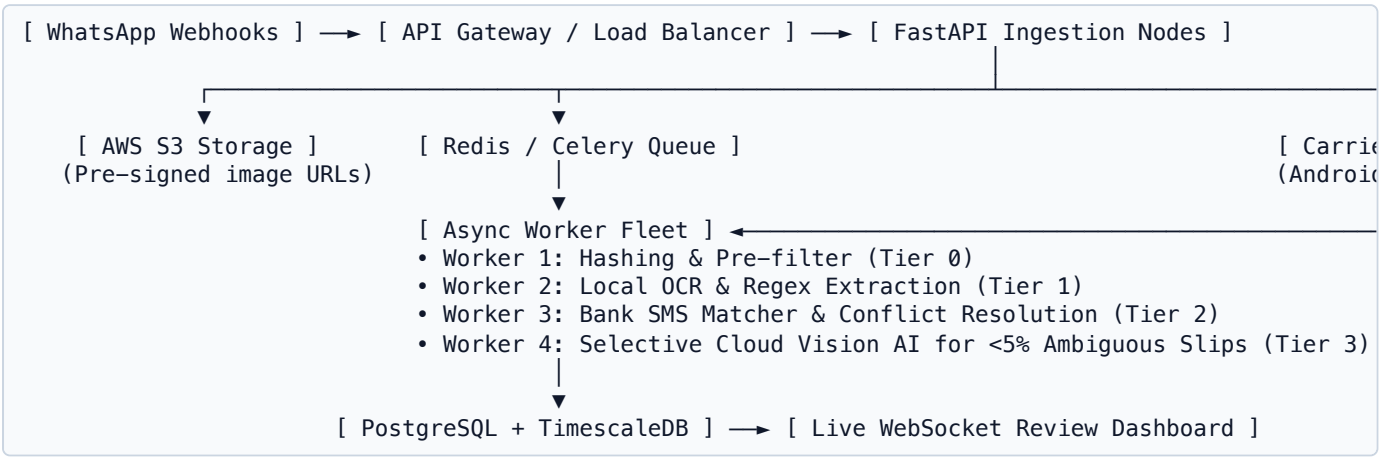
Pipeline Layer	Technology	Execution	Latency	API Cost / 1k Requests	Volume
Tier 0: Hash & Quality	SHA-256 + pHash + Pillow	Local CPU	$<5\text{ms}$	**$0.00**	100%
Tier 1: Local OCR	Tesseract OCR + Custom Regex	Local CPU	$\sim 50\text{ms}$	**$0.00**	~85%
Tier 2: SMS Correlator	Indexed SQL Query	Database	$<2\text{ms}$	**$0.00**	~80%
Tier 3: Vision AI (Fallback)	Gemini 1.5 Flash / Claude Haiku	Cloud API	$\sim 800\text{ms}$	**~$0.15** *(only if escalated)*	$<5\%$ (Ambiguous only)

Key Cost Strategy: Duplicates and invalid images are filtered at Tier 0 ($0 spend). Over 95% of standard slips are verified via local Tesseract and SMS correlation, yielding an effective blended external API spend of **$0.00 per 1,000 requests**.

7. Major Limitations

1. **Handwritten Deposit Slips**: Physical paper slips with cursive handwriting challenge traditional OCR; these safely fail over to `NEEDS_VERIFICATION` rather than falsely approving.
2. **Bank SMS Formatting Variations**: Different banks (BOC, Commercial Bank, HNB, Sampath) format SMS differently. Regexes must be maintained or backed by a lightweight NLP parser.
3. **SQLite Concurrency**: SQLite is single-writer; production requires PostgreSQL with connection pooling.
4. **Synchronous OCR Lifecycle**: In the prototype, OCR executes inside the HTTP request. High concurrency requires background workers.

8. Evolution for Production Scale (100,000+ Submissions/Month)



1. **Asynchronous Worker Queue**: WhatsApp webhooks return immediate `HTTP 200` receipts; Celery/Redis worker fleets process hashing, OCR, and SMS matching asynchronously.
2. **Encrypted Object Storage**: Slips stored in AWS S3 with time-limited pre-signed URLs; no files stored on web servers.
3. **Automated SMS Gateway**: Android SIM forwarders or aggregator webhooks push bank credit notifications into an encrypted message stream.
4. **Multi-Tenant PostgreSQL**: Multi-tenant database partitioning (`business_id`) and pgvector/LSH indexing for rapid perceptual hash lookups across millions of records.
5. **Human-in-the-Loop Operations Dashboard**: Live WebSocket interface enabling merchant support agents to review and resolve `NEEDS_VERIFICATION` cases with 1-click approvals.